

PE-EC703A — Embedded System

Semester VII • 3L:0T:0P • 3 Credits

Module 1 Exam Notes: Overview of Embedded Systems

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	What is an Embedded System?	2
1.1	Key Characteristics	2
2	Embedded Processor in a System	2
2.1	Types of Embedded Processors	3
3	Components of an Embedded System	3
3.1	Hardware Components	3
3.2	Software Components	4
4	Embedded Software in a System	4
4.1	Levels of Embedded Software	4
4.2	Brief Introduction to Embedded Software Categories	5
5	Design Process in Embedded Systems	5
5.1	Design Steps Explained	5
5.2	Hardware–Software Co-design	6

1. What is an Embedded System?

An embedded system is a combination of hardware and software designed to perform a specific dedicated function within a larger system. Unlike general-purpose computers, embedded systems are purpose-built and optimized for a particular task.

Embedded System — Definition

An **embedded system** is a dedicated computing system that is tightly integrated into the device it controls. It consists of a microprocessor or microcontroller, memory, I/O interfaces, and software (firmware) designed to perform one or more specific tasks — often with real-time constraints and limited resources (power, memory, processing speed).

1.1 Key Characteristics

- **Single-function:** Designed to perform a specific task (e.g., engine control, temperature monitoring).
- **Real-time operation:** Must respond to events within strict time deadlines.
- **Resource-constrained:** Limited memory (RAM/ROM), processing power, and energy budget.
- **Reliability:** Must operate continuously and correctly, often without human intervention.
- **Embedded into a product:** The computing part is hidden inside a larger product (e.g., washing machine, car).

2. Embedded Processor in a System

Embedded Processor

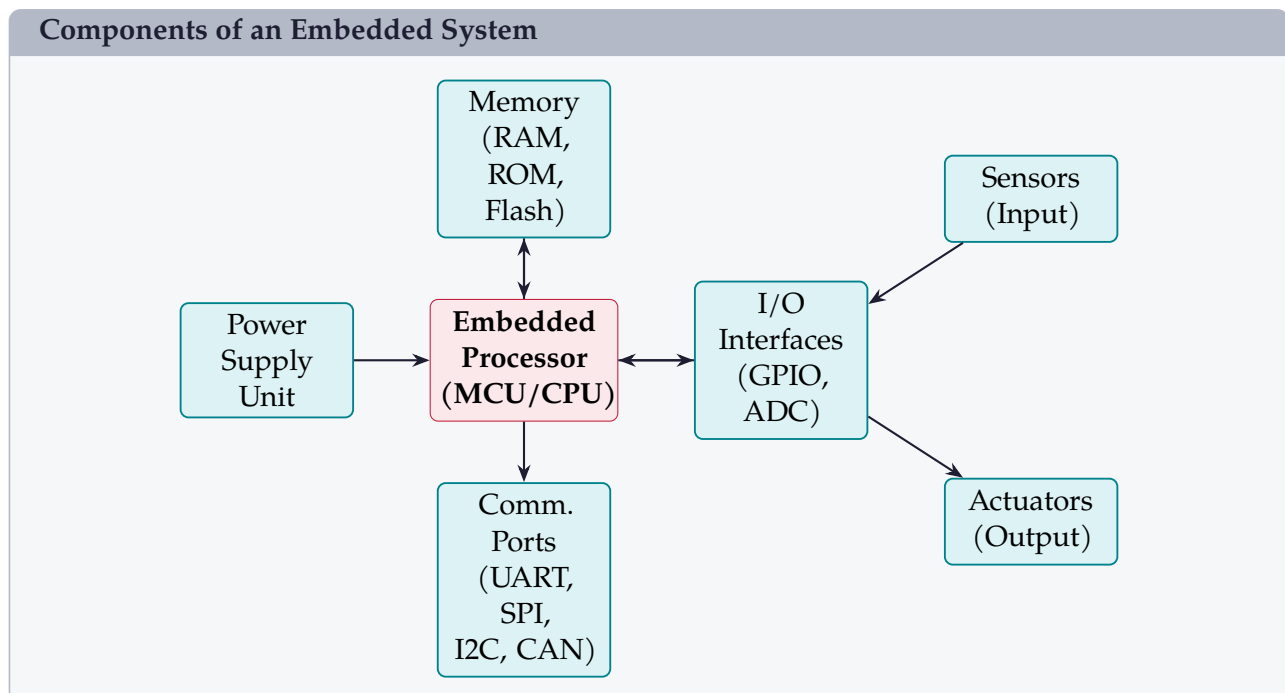
An **embedded processor** is the central computational element of an embedded system. It can be a **microprocessor** (requires external memory and peripherals), a **microcontroller** (integrates CPU, memory, and peripherals on a single chip), a **DSP** (optimized for signal processing), or an **FPGA/ASIC** (for high-speed, custom logic).

2.1 Types of Embedded Processors

Processor Type	Description	Examples / Use
Microcontroller (MCU)	CPU + RAM + ROM + I/O on one chip	8051, PIC, ARM Cortex-M, AVR
Microprocessor (MPU)	CPU only; needs external memory and peripherals	Intel 8086, ARM Cortex-A
DSP Processor	Optimized for multiply-accumulate (MAC) operations	TMS320, SHARC
FPGA / ASIC	Reconfigurable / custom-designed logic fabric	Xilinx, Intel Altera
SoC	Entire system (CPU + GPU + memory + I/O) on one die	Qualcomm Snapdragon, Raspberry Pi SoC

3. Components of an Embedded System

An embedded system is made up of several hardware and software components that work together.



3.1 Hardware Components

- **Processor Core:** The CPU that fetches, decodes, and executes instructions.
- **Memory:**
 - **ROM / Flash:** Stores firmware (non-volatile, permanent program code).
 - **RAM:** Temporary storage for variables and stack (volatile).

- **EEPROM:** Non-volatile, electrically erasable; stores configuration data.
- **I/O Ports:** Digital GPIO, analog ADC/DAC inputs and outputs.
- **Communication Interfaces:** UART, SPI, I2C, CAN, USB.
- **Timers/Counters:** Generate time delays, PWM signals, and measure frequency.
- **Clock Source:** Crystal oscillator, RC oscillator, or PLL — drives system timing.
- **Power Supply:** Regulated DC supply (often 3.3V or 5V); may include battery management.

3.2 Software Components

- **Boot Loader:** First code that runs on power-up; initializes hardware and loads the main program.
- **Operating System / RTOS:** Manages tasks, scheduling, memory, and I/O (optional; may be bare-metal).
- **Device Drivers:** Software layer that abstracts hardware peripherals (UART driver, SPI driver, etc.).
- **Application Software:** The main program implementing the embedded task.
- **Middleware:** Protocol stacks (TCP/IP, USB), file systems, or GUI libraries.

4. Embedded Software in a System

Embedded Software (Firmware)

Embedded software, also called **firmware**, is the software permanently programmed into the non-volatile memory (ROM/Flash) of an embedded system. It directly controls hardware and implements the system's application logic. Unlike desktop software, it cannot be easily updated by the end-user.

4.1 Levels of Embedded Software

The embedded software stack has distinct layers, each serving a different purpose:

Embedded Software Layered Architecture

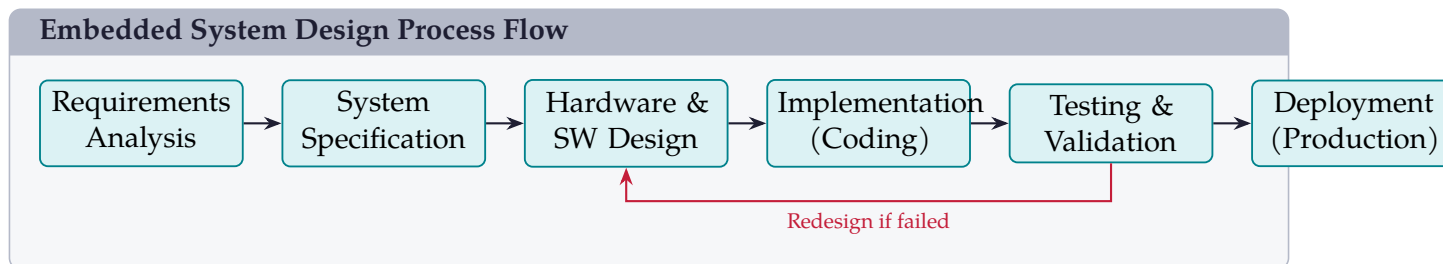
Application Software (Task-specific logic)
Middleware (Protocol stacks, File systems, GUI)
RTOS / Bare-metal Scheduler
Device Drivers (HAL — Hardware Abstraction Layer)
BSP / Boot Loader (Board Support Package)
Hardware (MCU, Memory, Peripherals)

4.2 Brief Introduction to Embedded Software Categories

- **Programming Languages Used:** Assembly language (for low-level hardware control), C (most common for embedded), and C++ (for OOP in embedded, limited use).
- **Bare-metal Programming:** No OS; application code runs directly on hardware. Used in simple, resource-constrained systems.
- **RTOS-based Programming:** Uses a real-time OS for multi-task management. Used when multiple functions must run concurrently with timing guarantees.
- **Firmware Updates (OTA):** Modern embedded systems support Over-The-Air updates to patch firmware without physical access.

5. Design Process in Embedded Systems

The development of an embedded system follows a structured design process to ensure the product meets functional, real-time, power, and cost requirements.



5.1 Design Steps Explained

1. **Requirements Analysis:** Identify what the system must do — functional requirements (what it does), non-functional requirements (speed, power, cost, size, reliability). Example: “Control motor speed based on temperature sensor; respond within 10 ms.”
2. **System Specification:** Define architecture — partitioning between hardware and software. Specify processor type, memory size, I/O count, communication protocol, and OS requirements.
3. **Hardware Design:** Schematic design (circuit), PCB layout, selection of MCU/processor, memory, power supply IC, and peripheral components.
4. **Software Design:** Algorithm design, data flow diagrams, state machines. Choosing programming language (C/Assembly), RTOS selection, ISR design.
5. **Implementation:** Writing firmware code (in C or Assembly). Cross-compilation using a host computer to generate binary for the target MCU. Flashing the binary into MCU flash memory.
6. **Testing and Validation:** Unit testing (individual modules), integration testing, hardware-in-the-loop (HIL) testing, real-time performance testing. Verify timing deadlines, power consumption, and EMI compliance.
7. **Deployment:** Mass production, product integration, final firmware version locked, field testing.

5.2 Hardware–Software Co-design

Modern embedded design uses **hardware–software co-design** — HW and SW are developed concurrently and their partitioning is optimized to meet cost and performance targets.

Aspect	Hardware Implementation	Software Implementation
Speed	Very fast (gate-level logic)	Slower (instruction-by-instruction)
Flexibility	Fixed after fabrication	Easily modified (just reflash)
Cost	High NRE (Non-Recurring Engineering)	Low cost for updates
Example	CRC calculation in FPGA	CRC computed in C function

Quick Revision — Module 1

Overview of Embedded Systems | PE-EC703A

Key Formulas at a Glance

System Clock Period	$T = \frac{1}{f_{clk}}$ (e.g., 12 MHz clock $\Rightarrow T = 83.3$ ns)
Instruction Cycle	$t_{inst} = \frac{N_{cycles}}{f_{clk}}$ (N_{cycles} = machine cycles per instruction)
Power Dissipation	$P = C \cdot V_{DD}^2 \cdot f$ (dynamic power in CMOS logic)
Response Time Deadline	$t_{response} \leq t_{deadline}$ (real-time constraint; must always hold)

One-Line Definitions

- **Embedded System:** Dedicated computing system embedded inside a product to perform a specific function with real-time constraints.
- **Firmware:** Software permanently stored in ROM/Flash; directly controls hardware.
- **MCU:** Microcontroller Unit — CPU, RAM, ROM, and peripherals all on one chip.
- **MPU:** Microprocessor Unit — CPU only; needs external support chips.
- **RTOS:** Real-Time Operating System — guarantees task completion within hard time deadlines.
- **HAL:** Hardware Abstraction Layer — software interface that hides hardware details from application code.
- **BSP:** Board Support Package — low-level software that initializes specific hardware board peripherals.
- **Bare-metal:** Programming directly on hardware without an operating system.
- **Cross-compiler:** A compiler running on a host PC that generates code for a different target architecture (e.g., ARM).
- **Hard real-time:** System where missing a deadline causes catastrophic failure (e.g., airbag control).
- **Soft real-time:** System where missing a deadline degrades performance but is not fatal (e.g., video streaming).

Frequently Asked / Viva Questions

Q1. Define an embedded system. How does it differ from a general-purpose computer?

An embedded system is a dedicated computing unit integrated into a product to perform a specific function (e.g., car ABS controller). A general-purpose computer (PC) can run any program. Embedded systems are

Frequently Asked / Viva Questions (contd.)

resource-constrained, single-function, and often real-time.

Q2. What are the key components of an embedded system?

- Processor (MCU/MPU), Memory (RAM/ROM/Flash), I/O Interfaces
- Communication ports (UART, SPI, I2C, CAN), Timers, Clock source, Power supply

Q3. What is the difference between a microprocessor and a microcontroller?

A **microprocessor** contains only the CPU; it needs external RAM, ROM, and peripherals. A **microcontroller** integrates CPU, RAM, ROM, and peripherals on a single chip — lower cost, smaller size, less power.

Q4. What is firmware? Where is it stored?

Firmware is the embedded software (program code) that directly controls hardware. It is stored in **non-volatile memory** — ROM, Flash, or EEPROM — so it persists after power-off.

Q5. What is meant by real-time in embedded systems?

Real-time means the system must complete its tasks within guaranteed time limits (deadlines). In **hard real-time** systems, missing a deadline causes system failure. In **soft real-time**, performance degrades but operation continues.

Q6. List the steps in the embedded system design process.

Requirements analysis → System specification → HW & SW design → Implementation (coding) → Testing and validation → Deployment.

Q7. What is hardware–software co-design?

It is the simultaneous design of hardware and software components, optimizing their partitioning to meet cost, performance, and power goals. Functions can be implemented in either HW (faster, fixed) or SW (flexible, cheaper to modify).

Q8. What is a BSP (Board Support Package)?

A BSP is a collection of low-level software routines specific to a hardware board. It initializes the processor, memory, clocks, and peripherals during boot, providing a foundation for higher-level software layers.

Q9. What is a cross-compiler and why is it needed?

A cross-compiler runs on a **host machine** (e.g., x86 PC) and generates executable code for a **target architecture** (e.g., ARM, 8051). It is needed because embedded targets often lack the resources to run a compiler themselves.

Q10. Give three examples of embedded systems in daily life.

Washing machine controller, automobile engine control unit (ECU), digital camera, ATM machine, and smart thermostat.

Frequently Asked / Viva Questions (contd.)

Q11. What is the role of a bootloader in an embedded system?

The bootloader is the first code executed after power-on. It initializes hardware (clocks, memory), and loads the main application firmware from flash into RAM (if needed) and transfers execution control to the application.

Q12. What is meant by a “resource-constrained” embedded system?

Resource-constrained means the system has very limited RAM (e.g., 256 bytes to 64 KB), flash (8 KB to 1 MB), processing power (8-bit, 8 MHz), and battery capacity. Firmware must be optimized to work within these tight constraints.

Comparison: Microprocessor vs. Microcontroller vs. DSP

Feature	Microprocessor	Microcontroller	DSP Processor
Integration	CPU only	CPU + RAM + ROM + I/O	CPU + MAC unit
External Parts	Many needed	Few or none	Some needed
Power	High	Low	Medium
Cost	High (system level)	Low (single chip)	Medium–High
Best For	Complex OS-based apps	Dedicated control tasks	Signal processing (audio, radar)
Example	Intel Core i7	PIC16F877, STM32	TI TMS320C6748